Offline Multi-modal RAG System for Privacy-Preserving Question Answering and Image Retrieval

Capstone Project Report submitted in partial fulfillment.
of the requirement for the degree of
B. E. (Information Technology)

Submitted By

22101B0001 Sanjana Kurade
22101B0008 Chinmay Tikole
22101B0016 Sanika Jade
22101B0030 Aniket Panchal

Under the Guidance of
Prof. Debarati Ghosal
Department of Information Technology

Vidyalankar Institute of Technology
Wadala(E), Mumbai 400 037
University of Mumbai

2025-26

CERTIFICATE OF APPROVAL

For

Capstone Project Report

This is to Certify that

Sanjana Kurade
Chinmay Tikole
Sanika Jade
Aniket Panchal

Have successfully carried out Capstone Project entitled

"Offline Multi-modal RAG System for Privacy-Preserving Question Answering and Image Retrieval"

In partial fulfillment of degree course in

Information Technology

As laid down by University of Mumbai during the academic year 2025-26

Under the Guidance of

Prof. Debarati Ghosal

Signature of Guide

Examiner 1 Examiner 2

Table of Contents

Sr. No.	Topic	Page No.
1.	Introduction	1
2.	Aim & Objectives	2-3
3.	Literature Review	4
4.	Methodology	5-9
5.	Proposed System	10-12
6.	Implementation	13-18
7.	Results and Discussion	19
8.	Conclusion	20
9.	Future Scope	21
10.	References	22

List of Figures

Sr. No.	Figure	Page No.
1	System Architecture	11
2	Implementation Screenshots	16
3	Implementation Screenshots	17
4	Implementation Screenshots	17
5	Implementation Screenshots	18
6	Implementation Screenshots	18

1. Introduction

In today's AI-driven world, information retrieval systems are evolving beyond text to incorporate multiple data types such as images, documents, and structured content. This project presents an Offline Multimodal Retrieval-Augmented Generation (RAG) System built using FastAPI, FAISS, and locally hosted LLM models. The system is designed with a privacy-first architecture, ensuring that all document processing, embedding generation, and inference occur entirely on the user's local machine without any data being transmitted to external servers.

The system enables users to query a knowledge base through both text and image inputs, retrieve semantically relevant information, and generate intelligent, context-aware responses in natural language. A key advancement over conventional RAG implementations is the integration of multilingual support via the paraphrase-multilingual-MiniLM-L12-v2 sentence transformer model, which supports over 50 languages and allows users to submit queries in their native language.

The core of the system lies in the combination of Sentence Transformer embeddings for semantic text representation and FAISS for efficient similarity search across large-scale vector databases. The ingestion pipeline processes PDFs into text chunks and extracts embedded images with surrounding contextual text, which are together stored and indexed for fast retrieval. Additionally, the system features an offline voice input module powered by faster-whisper, enabling speech-to-text transcription without any internet dependency after the initial one-time model download.

When a user submits a query, the system identifies the intent (textual, visual, or combined) using multilingual keyword-based heuristics across 12 languages, fetches the top-k most relevant documents, and leverages a local LLM (served via Jan AI) to provide an enriched and coherent response grounded in retrieved context. Answers include clickable source citations linking back to the originating PDF documents.

The backend, developed in FastAPI, ensures high performance and easy scalability through RESTful endpoints with asynchronous background ingestion. The frontend, built with HTML, CSS, and JavaScript (Jinja2 templates), provides a clean and interactive user interface where users can upload files organised in topic-based folders, enter queries, use voice input, and visualize the system's responses seamlessly.

By combining retrieval and generation with multilingual and multimodal understanding, this project demonstrates a practical implementation of advanced AI techniques for privacy-preserving knowledge augmentation, research assistance, and intelligent search systems — all without requiring an internet connection during operation.

2. Aim and Objectives

Aim

The main aim of this project is to develop an Offline Multimodal Retrieval-Augmented Generation (RAG) System that can understand and process both text and image inputs to deliver intelligent, context-aware, and meaningful responses — entirely on the user's local machine with no data leaving the device. The system combines the strengths of information retrieval and generative AI using tools like FastAPI, FAISS, multilingual sentence embeddings, and a locally hosted LLM.

By integrating these technologies, the project seeks to create a privacy-preserving AI assistant capable of understanding multilingual queries, supporting offline voice input, retrieving relevant data and images efficiently, and generating accurate human-like responses with source citations. This approach bridges the gap between traditional text-based search engines and advanced multimodal, multilingual AI systems, making it suitable for use in education, research, healthcare, legal domains, and enterprise-level knowledge management.

Objectives

1. Multimodal Data Ingestion

- Design a pipeline to handle text documents and PDFs, extracting both text chunks and embedded images.
- Use multilingual sentence transformer embeddings (paraphrase-multilingual-MiniLM-L12-v2) to convert text data into 384-dimensional vector representations.
- Extract images from PDFs using PyMuPDF, capturing surrounding contextual text (captions, nearby paragraphs) for semantic image description.
- Organise uploaded PDFs into named category subfolders (e.g., biology, company-docs) for structured document management.
- Support incremental document management — adding or deleting PDFs triggers automatic re-ingestion without manual intervention.

2. Efficient Retrieval using FAISS

- Implement FAISS for high-speed similarity search across large vector databases.
- Retrieve top-k relevant items (k=15, top 5 text chunks for LLM context) efficiently for any given text or image query.
- Store both text and image embeddings in a unified FAISS index for consistent retrieval.

3. Backend Development using FastAPI

- Build a FastAPI-based backend to manage requests for ingestion, retrieval, response generation, file management, and analytics.
- Implement asynchronous background ingestion so PDF upload and processing do not block the user interface.
- Provide secure API endpoints including path-traversal guards for PDF serving.
- Expose additional endpoints for analytics (/analytics), security auditing (/security-audit), PDF preview (/pdf-preview), and health monitoring (/health).

4. Multilingual Query Support

- Integrate language detection via langdetect to identify the user's query language.
- Implement multilingual intent analysis using keyword dictionaries spanning 12 languages (English, Spanish, French, German, Portuguese, Italian, Dutch, Russian, Chinese, Japanese, Korean, Arabic).
- Enable users to ask questions in any language and receive relevant responses through the multilingual MiniLM embedding model.

5. Offline Voice Input

- Integrate faster-whisper for fully offline speech-to-text transcription using a locally cached Whisper base model (~145 MB).
- Allow users to submit queries by voice through a microphone button in the chat interface, with transcription occurring entirely on-device after the first-time model download.
- Implement SpeechRecognition/pyaudio as optional dependencies with graceful fallback if unavailable.

6. Intelligent Response Generation with Local LLM

- Use Jan AI's locally hosted LLM (OpenAI-compatible API at http://localhost:1337/v1) to generate responses based on retrieved context.
- Combine retrieved text context with the user query in a structured prompt to minimise hallucination.
- Include clickable source citations in responses, referencing the originating PDF document and page number.

7. User Interface and Interaction

- Develop a Home page (/) for PDF upload, folder organisation, file listing, and deletion.
- Develop a Chat page (/chat) with interactive messaging, typing indicators, dynamic input resizing, and voice input support.
- Display retrieved images in a grid with clickable full-screen preview overlays.
- Redirect users to the Home page if no embeddings are available when accessing the Chat page.

8. Evaluation and Optimization

- Measure system performance using retrieval accuracy, response relevance, and latency metrics.
- Use batch embedding computation and manifest-based change tracking to efficiently manage large document sets.
- Perform full vector store rebuild on any document change to ensure consistency.

3. Literature Review

The integration of Retrieval-Augmented Generation (RAG) has transformed the way AI systems handle information retrieval and response generation. Traditional language models like GPT and BERT generate fluent text but often lack factual grounding, leading to hallucinated responses. To address this, Lewis et al. (2020) introduced RAG, combining retrieval-based search with generation models to produce contextually accurate and factually grounded responses.

Recent research has extended RAG to multilingual and multimodal systems. The introduction of multilingual sentence transformers — particularly the paraphrase-multilingual-MiniLM-L12-v2 model developed by Reimers and Gurevych (2019) and extended by the Sentence Transformers library — established efficient cross-lingual embedding representations, enabling semantic search across over 50 languages in a unified vector space. This is a significant departure from earlier systems that required separate models per language.

Earlier multimodal work leveraged CLIP (Contrastive Language-Image Pre-training) by OpenAI (2021), which established a shared embedding space for images and text. While powerful, CLIP embeddings are limited to English text-image matching. The current system adopts a text-centric approach to image retrieval: images are embedded via their extracted surrounding textual context using the multilingual MiniLM model, making image retrieval accessible across languages without requiring raw pixel-level embedding.

For vector storage and similarity search, FAISS (Facebook AI Similarity Search) by Meta AI remains a standard tool, offering efficient large-scale vector indexing and fast nearest-neighbor searches on both CPU and GPU. Integrating FAISS with multilingual sentence embeddings enables rapid retrieval of relevant content across diverse, multilingual document collections.

Offline speech recognition has been advanced significantly by OpenAI's Whisper model family, with faster-whisper providing an optimised CTranslate2-based implementation suitable for local deployment. This aligns with the growing demand for privacy-preserving AI applications where audio data must not leave the user's device.

Frameworks like FastAPI have further simplified the deployment of AI models, offering high performance, async processing, and easy API integration. Projects such as LangChain and LlamaIndex provide structured pipelines for retrieval and generation tasks. This project leverages LangChain's PyPDFLoader and RecursiveCharacterTextSplitter for document ingestion while building custom intent analysis and multilingual query routing on top.

This project builds upon these advancements by combining multilingual FAISS-based retrieval, contextual image extraction via PyMuPDF, offline Whisper voice input, and local LLM inference into a unified privacy-preserving RAG system — extending RAG capabilities to multilingual, multimodal, and voice-enabled real-world knowledge management scenarios.

4. Methodology

The methodology outlines the systematic approach followed in designing, developing, and implementing the Offline Multimodal RAG System. This project combines information retrieval, multilingual natural language processing, offline speech recognition, and local language model inference to create a fully privacy-preserving AI assistant capable of understanding text and image inputs across multiple languages.

The methodology consists of several key phases: Data Ingestion, Multilingual Embedding Generation, Vector Indexing using FAISS, Query Processing with Intent Detection, Response Generation using a local LLM, Backend Integration via FastAPI, and Frontend Development with voice input.

1. System Design Overview

The proposed system architecture is divided into three major layers:

1. Data Layer: Responsible for data ingestion, PDF upload organisation, and storage of extracted text chunks and images.
2. Processing Layer: Handles multilingual feature extraction, embedding generation, and FAISS-based similarity search.
3. Application Layer: Manages user interaction, multilingual query handling, offline voice input, and response generation using FastAPI and a local LLM.

Each layer communicates through well-defined interfaces, ensuring modularity, scalability, and ease of maintenance.

2. Data Ingestion and Preprocessing

The first step in building the RAG system is the ingestion of PDF-based data. Users upload PDFs via the Home page, organised into named category subfolders for structured management.

2.1 Data Collection

Users upload PDF documents or entire folders through the frontend interface at the /upload-files endpoint. Files are saved under uploads/<category>/ subfolders. Upload automatically triggers background ingestion via FastAPI's BackgroundTasks, so the user interface remains responsive during processing.

2.2 Text Extraction

For textual data, the system uses LangChain's PyPDFLoader and RecursiveCharacterTextSplitter to load and chunk PDFs into segments of approximately 500 characters with 50-character overlap. Each text chunk retains metadata including the source PDF path, filename, PDF name stem, and page number for source citation generation.

2.3 Image Extraction with Contextual Text

For image data, the system uses PyMuPDF (fitz) to extract all embedded images from each PDF page. For each image, the system captures surrounding contextual text using bounding box expansion with a 50-pixel margin. If no text is found in the immediate surrounding area, a page-level text fallback captures the first 500 characters of page text. This contextual text, combined with a page-level description, forms the image's semantic representation for

embedding — enabling multilingual image retrieval without raw pixel embeddings. Images are saved to the images/ directory with descriptive filenames (e.g., biology_p3_img1.png).

3. Multilingual Embedding Generation

To enable multilingual semantic understanding, the system uses the paraphrase-multilingual-MiniLM-L12-v2 model from the Sentence Transformers library for embedding generation. This model was selected for its balance of efficiency and multilingual coverage.

3.1 Model Characteristics

- Dimensionality: 384-dimensional dense vectors
- Language Support: 50+ languages including English, Hindi, Spanish, French, German, Japanese, Chinese, Arabic, and more
- Model Size: Approximately 100 MB, lightweight and suitable for local deployment
- Download: Cached by HuggingFace on first run; fully offline thereafter

3.2 Embedding Process

- Each text chunk and image description is converted into a 384-dimensional embedding.
- Embeddings are normalised for consistent cosine similarity computation in FAISS.
- Both text and image embeddings are stored in the same FAISS index, enabling unified retrieval across modalities.

This approach replaces the earlier CLIP-based embedding (clip-ViT-B-32) with a more versatile multilingual model, significantly improving retrieval quality for non-English queries.

4. Vector Indexing and Retrieval using FAISS

After generating embeddings, the system stores them in a FAISS index for fast similarity search.

4.1 Index Creation

- The PDFIngestor class in ingest.py performs a full rebuild of the vector store on every ingestion run, deleting the previous FAISS index and images directory to ensure consistency.
- A FAISS index is built using FAISS.from_documents() with the multilingual embeddings, and saved to vector_store/ as index.faiss and index.pkl.
- A manifest.json file tracks ingested PDF paths and their modification timestamps, supporting change detection for future re-ingestion.

4.2 Search Mechanism

When a user submits a query, its embedding is generated using the same multilingual model. FAISS then performs similarity_search_by_vector with k=15, returning the top-15 most semantically similar entries. Of these, the top 5 text chunks are used as context for the LLM, and up to 6 relevant images (deduplicated) are included in the visual response.

5. Query Intent Analysis and Multilingual Routing

A key feature of the system is its ability to determine whether the user wants a text answer, image retrieval, or both — and to do so across multiple languages.

5.1 Language Detection

The system uses langdetect to identify the language of the incoming query. This is implemented with a graceful fallback if langdetect is unavailable, defaulting to English-based intent detection.

5.2 Multilingual Intent Classification

The analyze_query_intent() function in app.py uses keyword dictionaries spanning 12 languages to classify each query into one of three intent categories:

- text_only: The user wants a textual answer (keywords: explain, describe, what is, define, etc. in all supported languages)
- images_only: The user wants images or diagrams (keywords: show image, diagram, picture, etc. in all supported languages)
- both: The user wants both text and images

This multilingual intent detection ensures that users querying in Hindi, Spanish, Japanese, Arabic, or any other supported language receive correctly routed responses.

5.3 Query Enhancement

Before retrieval, queries are enhanced with additional contextual terms (e.g., appending visual content, diagram for image-intent queries) to improve FAISS retrieval quality.

6. Offline Voice Input

The system includes an offline voice input feature powered by faster-whisper, enabling users to submit queries by voice without internet connectivity.

- On first use, the Whisper base model (~145 MB) is downloaded from HuggingFace and cached in the whisper_model/ directory.
- Subsequent transcriptions are fully offline.
- The model is loaded using WhisperModel('base', device='cpu', download_root=...) with compute_type='int8' for efficiency.
- Voice input is accessible via a microphone button in the chat interface.
- SpeechRecognition and pyaudio are optional dependencies — the app runs without them, and the voice button is gracefully disabled if they are absent.

Note: Python 3.10-3.12 is required for voice input, as the audioop module used by SpeechRecognition was removed in Python 3.13.

7. Response Generation using Local LLM

Once the most relevant documents are retrieved, the system uses a locally hosted LLM (via Jan AI) to produce a coherent, natural language response.

7.1 LLM Integration

The system uses the openai Python library pointed at a local Jan server running at http://localhost:1337/v1 (OpenAI-compatible API). This means no data is ever sent to OpenAI's servers — all inference happens locally.

7.2 Prompt Construction

Retrieved text snippets are concatenated into a context block and passed to the LLM with the following structured prompt:

```
Based on the context below, provide a clear and accurate
answer to the question. Be concise. Context: {retrieved
context} Question: {user question} Answer:
```

7.3 Source Citations

The /ask endpoint returns clickable citation HTML in the response, referencing the originating PDF document and page number. The /serve-pdf endpoint serves PDF files via HTTP, enforcing path-traversal protection by validating that all file paths begin with uploads/.

8. Backend Implementation using FastAPI

The FastAPI framework serves as the backbone of the application, managing all communication between the frontend, FAISS index, and local LLM.

8.1 API Endpoints

Endpoint	Description
/ and /home	File management UI — upload, list, delete PDFs
/chat	Chat interface — redirects to home if vector store is empty
/ask (POST)	Main query endpoint — intent analysis, retrieval, LLM generation
/upload-files	Handles PDF uploads with background ingestion trigger
/list-files	Lists uploaded PDFs by category folder
/delete-file	Deletes a PDF and triggers re-ingestion
/run-ingestion	Manually triggers full vector store rebuild
/serve-pdf	Serves PDF files with path-traversal protection
/pdf-preview	Returns PDF preview data
/images/{name}	Serves extracted PDF images
/health	System health check (vector DB, LLM, image count)
/analytics	Usage analytics endpoint
/security-audit	Security audit endpoint

8.2 Asynchronous Processing

FastAPI's BackgroundTasks enable PDF ingestion to run in the background while the user interface remains responsive. Multiple queries are handled concurrently through FastAPI's async request handling.

9. Frontend Interface and User Interaction

The frontend is built with HTML, CSS, and JavaScript using Jinja2 templates, providing two primary interfaces:

9.1 Home Page (/)

- PDF upload form with category/folder name input
- Uploaded files listing with per-file delete functionality
- Manual ingestion trigger button
- Navigation to the Chat interface

9.2 Chat Page (/chat)

- Interactive chat message display (user vs AI)
- Typing indicator while the LLM generates a response
- Resizable textarea for multi-line query input
- Microphone button for offline voice input
- Image grid for displaying retrieved images
- Full-screen image preview overlay with click-to-enlarge
- Clickable source citations linking back to originating PDFs
- Smooth scrolling and responsive design

5. Proposed System

System Architecture

The proposed system consists of several interconnected components that work together to provide accurate multimodal responses with multilingual support and full offline privacy:

1. Input Processing Module
   - Accepts queries in text, voice, or combined formats in 50+ languages
   - Performs preprocessing including text tokenisation and language detection
   - Converts voice input to text using the offline Whisper model
2. Feature Extraction Module
   - Generates 384-dimensional embeddings for text data using paraphrase-multilingual-MiniLM-L12-v2
   - Converts PDF images into searchable entries via surrounding contextual text extraction and multilingual embedding
   - Combines all embeddings into a unified FAISS vector store
3. Retrieval Module
   - Performs FAISS similarity search (k=15) against the indexed knowledge base
   - Routes results to text context (top 5 chunks) and image grid (up to 6 images) based on intent
4. Generative Response Module
   - Processes retrieved context with the user query using a local LLM via Jan AI
   - Produces coherent, context-grounded, human-like responses
   - Includes clickable source citations for retrieved documents
5. File Management Module
   - Manages PDF upload, organisation by category, listing, and deletion
   - Triggers automatic background re-ingestion on document changes
6. User Interface Module
   - Home page for document management and ingestion status
   - Chat page for query submission, voice input, text+image response display, and citation viewing

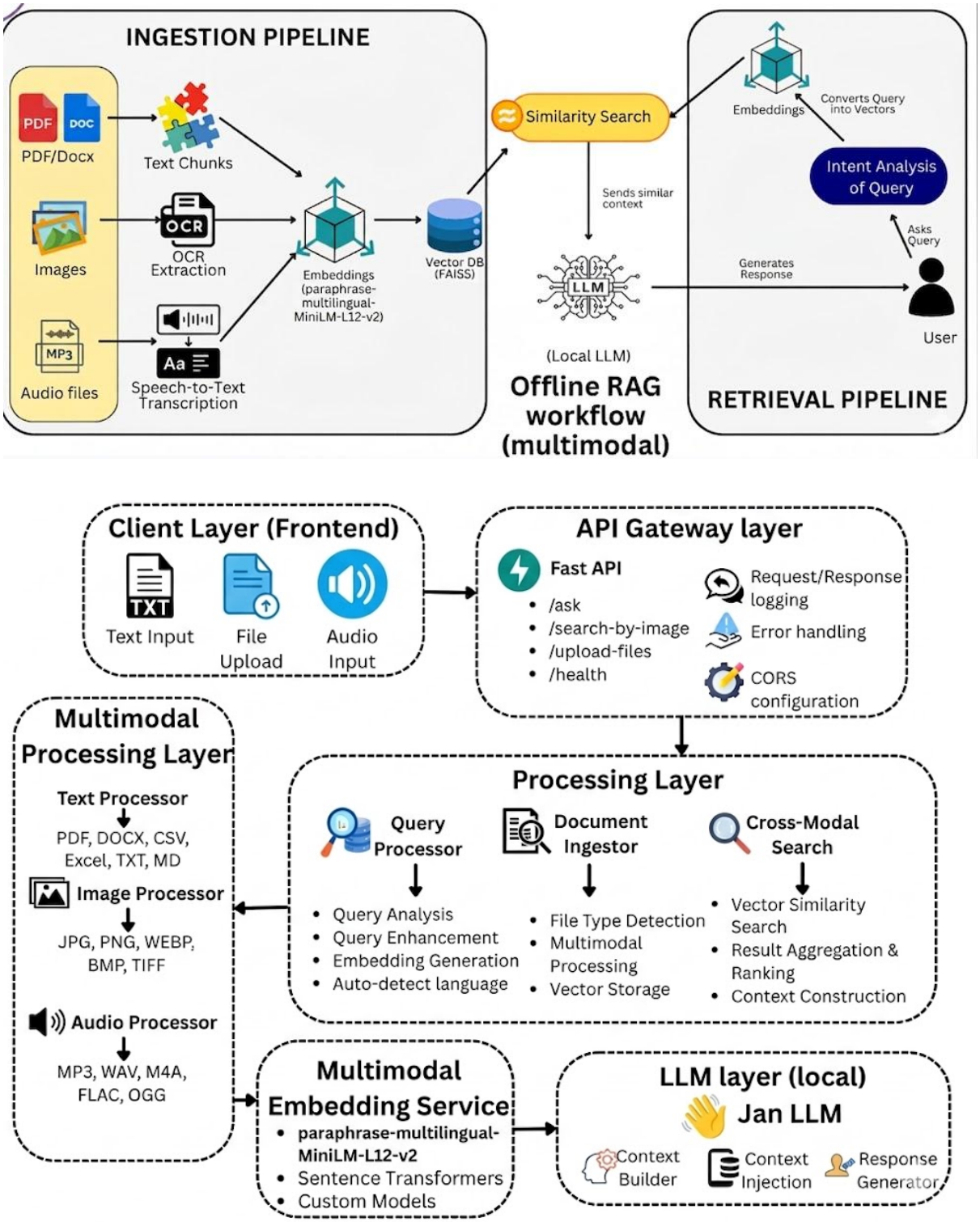

Figure 1: System Architecture

Hardware Requirements

- Processor: Intel Core i5 or equivalent (i7 recommended for faster embedding computation)
- RAM: Minimum 8 GB (16 GB recommended for large model inference and large document sets)

- Storage: Minimum 10 GB free space (additional space for PDF uploads, extracted images, and locally cached models)
- Graphics Card: NVIDIA GPU (optional but recommended for faster embedding and Whisper inference; CPU mode is supported)

Software Requirements

- Operating System: Windows 10/11, macOS, or Linux
- Python Version: 3.10-3.12 (Python 3.13 is not supported due to removal of audioop)
- Local LLM Server: Jan AI or LM Studio (OpenAI-compatible API at http://localhost:1337/v1)

Libraries and Frameworks:

Package	Purpose
fastapi + uvicorn	Web server and async API framework
langchain + langchain-community	LLM orchestration, document loading, text splitting
faiss-cpu / faiss-gpu	Vector similarity search
sentence-transformers	Multilingual text embeddings (MiniLM)
PyMuPDF (fitz)	PDF text and image extraction
faster-whisper	Offline speech-to-text transcription
Pillow	Image processing
langdetect	Query language detection
openai	OpenAI-compatible client for local LLM
SpeechRecognition / pyaudio	Voice input (optional)
NumPy, Pandas	Data processing
Jinja2	HTML template rendering

Storage Layout

Directory / File	Contents
uploads/<category>/	PDFs organised in named topic subfolders
images/	Extracted PDF images (e.g., biology_p3_img1.png)
vector_store/	FAISS index files (index.faiss, index.pkl)
vector_store/manifest.json	Tracks ingested PDFs by path and modification time
whisper_model/	Cached Whisper base model (~145 MB)
templates/	Jinja2 HTML templates (home.html, index.html)

6. Implementation

1. Overview of the System

This system is a fully offline, privacy-preserving document-based AI assistant that can:

- Answer textual questions about uploaded PDF documents in multiple languages
- Retrieve relevant images extracted from PDFs based on semantic similarity
- Provide combined text and image responses for comprehensive answers
- Accept voice queries offline using the locally cached Whisper model
- Display source citations linking answers back to the originating PDF and page

It is built using:

- FastAPI for the backend server with async background task processing
- FAISS for semantic vector search over document embeddings
- Sentence Transformers (paraphrase-multilingual-MiniLM-L12-v2) for multilingual text and image-context embeddings
- Jan AI (local LLM server) for generating natural language answers via OpenAI-compatible API
- faster-whisper for offline speech-to-text transcription
- HTML/CSS/JS frontend (Jinja2 templates) for interactive chat and file management

2. Project Structure

```
offline_RAG_chatbot/ ├── app.py              # FastAPI application (main
entry point) ├── ingest.py           # PDF ingestion & FAISS vector
store builder ├── requirements.txt    # Python dependencies ├──
CLAUDE.md           # Architecture and developer documentation ├──
test.py             # CLI tool for querying vector store ├── bt_print.py
# Serial port utility (unrelated to RAG pipeline) ├── templates/ │   ├──
home.html       # File management UI │   └── index.html      # Chat
interface ├── uploads/            # PDFs organised in category
subfolders ├── vector_store/       # Auto-generated FAISS index +
manifest.json ├── images/             # Auto-extracted PDF images └──
whisper_model/      # Cached Whisper model (first-run download)
```

3. Components

3.1 Backend (app.py)

app.py is the main API service handling the server lifecycle, all API endpoints, query processing, and LLM integration. On startup (lifespan), it loads the FAISS vector store and initialises the local LLM client (openai.OpenAI pointed at http://localhost:1337/v1). Resources are cleaned up on shutdown.

Key query processing steps in the /ask endpoint:

7. Intent analysis: Determines if the query wants text_only, images_only, or both using multilingual keyword dictionaries (12 languages).
8. Language detection: Identifies query language via langdetect with graceful fallback.
9. Query enhancement: Appends contextual terms to improve retrieval for image-intent queries.
10. Vector search: Uses FAISS similarity_search_by_vector (k=15) to find semantically similar entries.

11. LLM generation: Sends top 5 text chunks as context to the local Jan LLM and generates a concise, context-aware answer.
12. Citation formatting: Includes HTML-formatted clickable citations referencing source PDFs and pages.
13. Image assembly: Selects up to 6 deduplicated relevant images and formats them as an HTML grid.

3.2 Vector Store (FAISS) and Embeddings

- FAISS stores unified embeddings for both text chunks and image descriptions.
- Embeddings are generated using paraphrase-multilingual-MiniLM-L12-v2 (384 dimensions), replacing the earlier clip-ViT-B-32 model.
- FAISS performs cosine similarity search to retrieve semantically relevant entries across languages.
- The index is rebuilt from scratch on every ingestion run — no stale entries are left in the vector store.

3.3 Document Ingestion (ingest.py)

The PDFIngestor class handles the complete pipeline from PDF to FAISS index:

14. PDF loading and text splitting: Uses PyPDFLoader and RecursiveCharacterTextSplitter (chunk_size=500, chunk_overlap=50) to split PDFs into manageable text chunks with page metadata.
15. Image extraction with context: Uses PyMuPDF to extract images. For each image, surrounding text is captured within a 50-pixel bounding box margin. Page-level text (first 500 characters) is used as fallback context. Images are saved to images/ with descriptive filenames.
16. Full rebuild: full_rebuild() always deletes the existing vector_store/ and images/ directories before processing, ensuring a clean and consistent index.
17. Manifest tracking: manifest.json saves the current state of ingested PDFs (paths and modification times) for future change detection.
18. Unified vector store: Both text chunks and image description documents are embedded using the multilingual model and stored in a single FAISS index.

3.4 Frontend (home.html and index.html)

The Home page (home.html) provides:

- Folder/category name input and multi-file PDF upload form
- Uploaded files listing per category with per-file delete buttons
- Manual ingestion trigger with status feedback
- Navigation link to the Chat interface

The Chat page (index.html) provides:

- Interactive chat message display (user vs AI alignment)
- Typing indicator while LLM is generating
- Resizable textarea for query input
- Microphone button for offline voice query submission
- Image grid for displaying retrieved PDF images
- Full-screen image preview overlay with click-to-zoom functionality

- Clickable source citations in AI responses, linking to the originating PDF via /serve-pdf
- AJAX-based communication with the /ask endpoint (no page reloads)

3.5 Offline Voice Input

Voice input is implemented using faster-whisper with a locally cached base model. The user clicks the microphone button in the chat interface, which records audio and submits it for transcription. The transcribed text is then submitted as a query. The Whisper model is cached in whisper_model/ to avoid Windows symlink permission issues and to ensure fully offline operation after the initial download.

3.6 Multilingual Query Intent and Context Handling

Query intent analysis: The analyze_query_intent() function classifies queries into text_only, images_only, or both using keyword dictionaries for 12 languages (English, Spanish, French, German, Portuguese, Italian, Dutch, Russian, Chinese, Japanese, Korean, Arabic). Each language has dedicated image-retrieval keywords and text-retrieval keywords.

Query enhancement: Image-intent queries are augmented with terms like visual content, diagram to improve image retrieval from FAISS.

Context selection: Top 5 text chunks are used for LLM context; up to 6 deduplicated images are selected for the visual response.

3.7 Source Citations

Each /ask response includes HTML-formatted clickable citations. Citations reference the source PDF filename and page number, and link to /serve-pdf which serves the PDF file directly. The /serve-pdf endpoint enforces a path-traversal guard, validating that all file paths begin with uploads/ before serving.

4. Workflow Summary

Step 1: Document Ingestion

19. User uploads PDFs via the Home page, organised into named category subfolders.
20. Background ingestion triggers automatically: text chunks extracted, images extracted with contextual text.
21. All embeddings generated using multilingual MiniLM model and stored in FAISS.

Step 2: User Query

22. User types a query in any supported language, or uses voice input via the microphone button.
23. Backend detects language, analyses intent (text/images/both) using multilingual keywords.
24. Query is enhanced and embedded using the multilingual model.
25. FAISS searches for top-15 semantically similar entries.

Step 3: Response Generation

26. Top 5 text chunks sent to local Jan LLM as context; text answer generated.
27. Up to 6 relevant images selected and deduplicated.
28. Combined HTML response formatted with text, citations, and image grid.

Step 4: Frontend Display

29. Response injected into chat interface with smooth scrolling.
30. Images clickable — full-screen overlay view on click.
31. Source citations clickable — open originating PDF in browser.
32. Typing indicators, dynamic input resizing, and responsive design throughout.

Implementation Screenshots

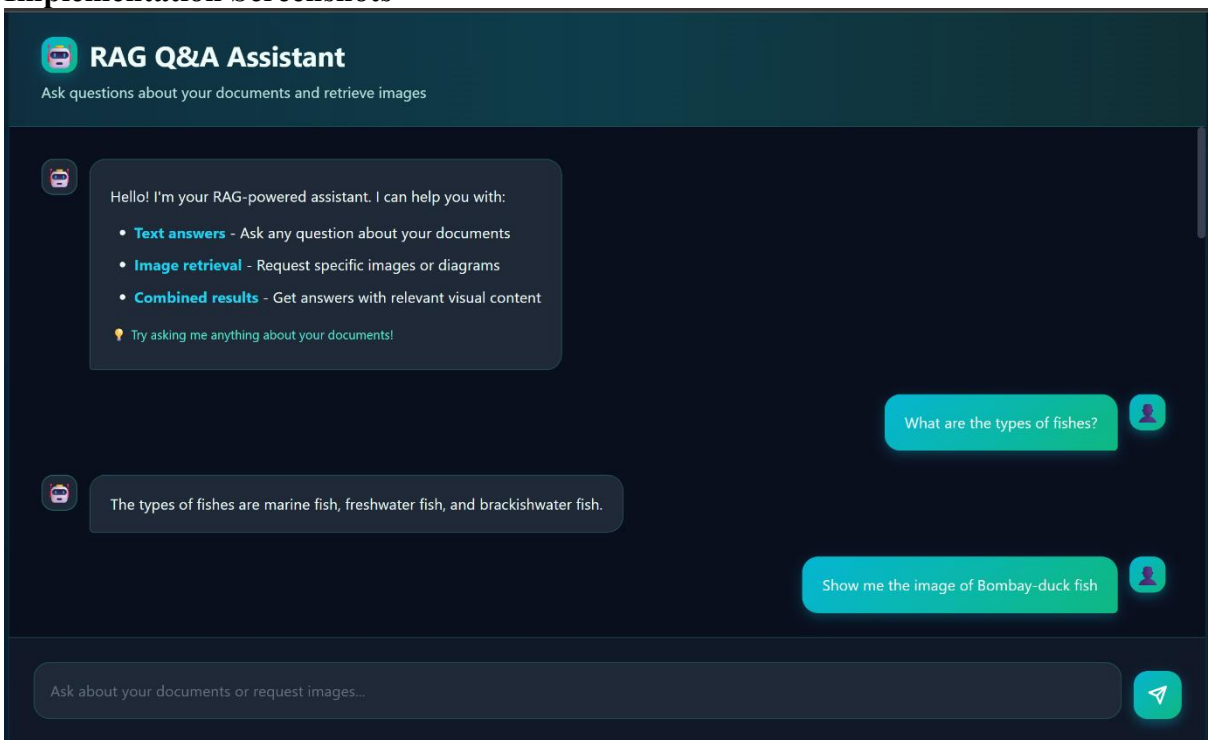

Figure 2: Implementation Screenshots

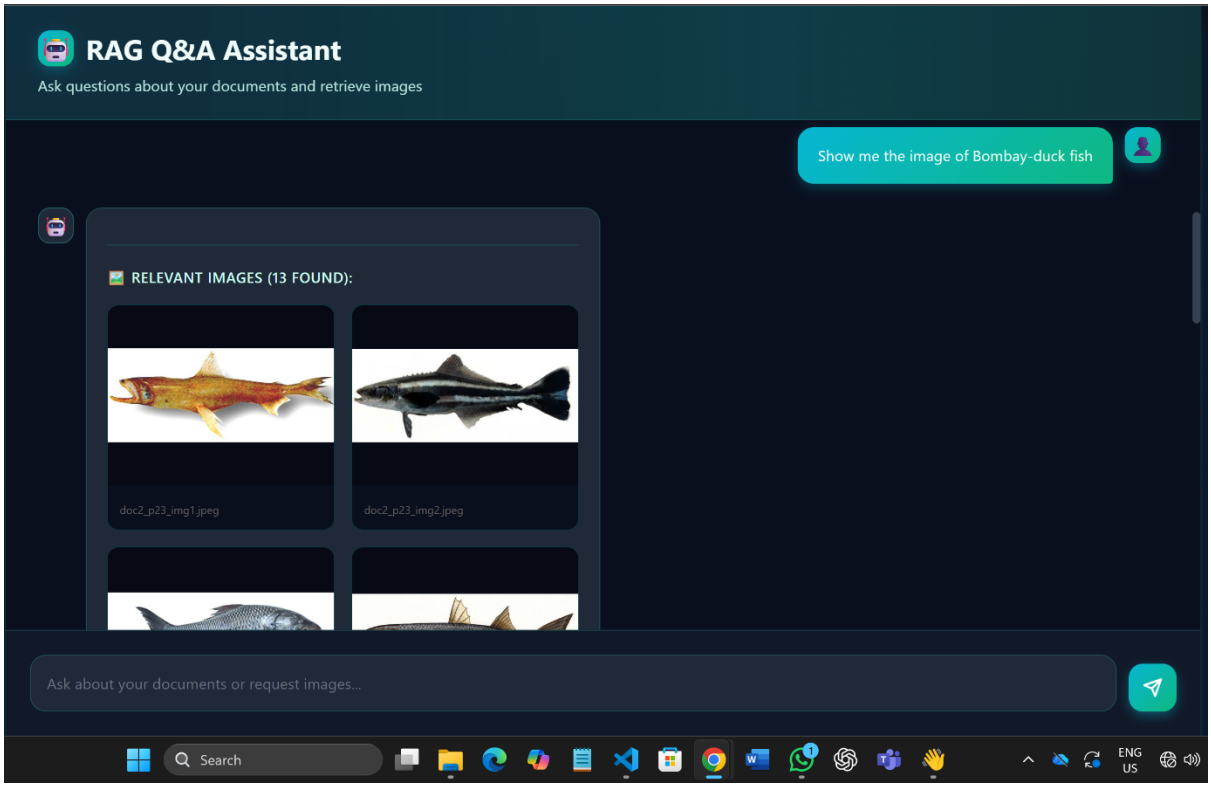

Figure 3: Implementation Screenshots

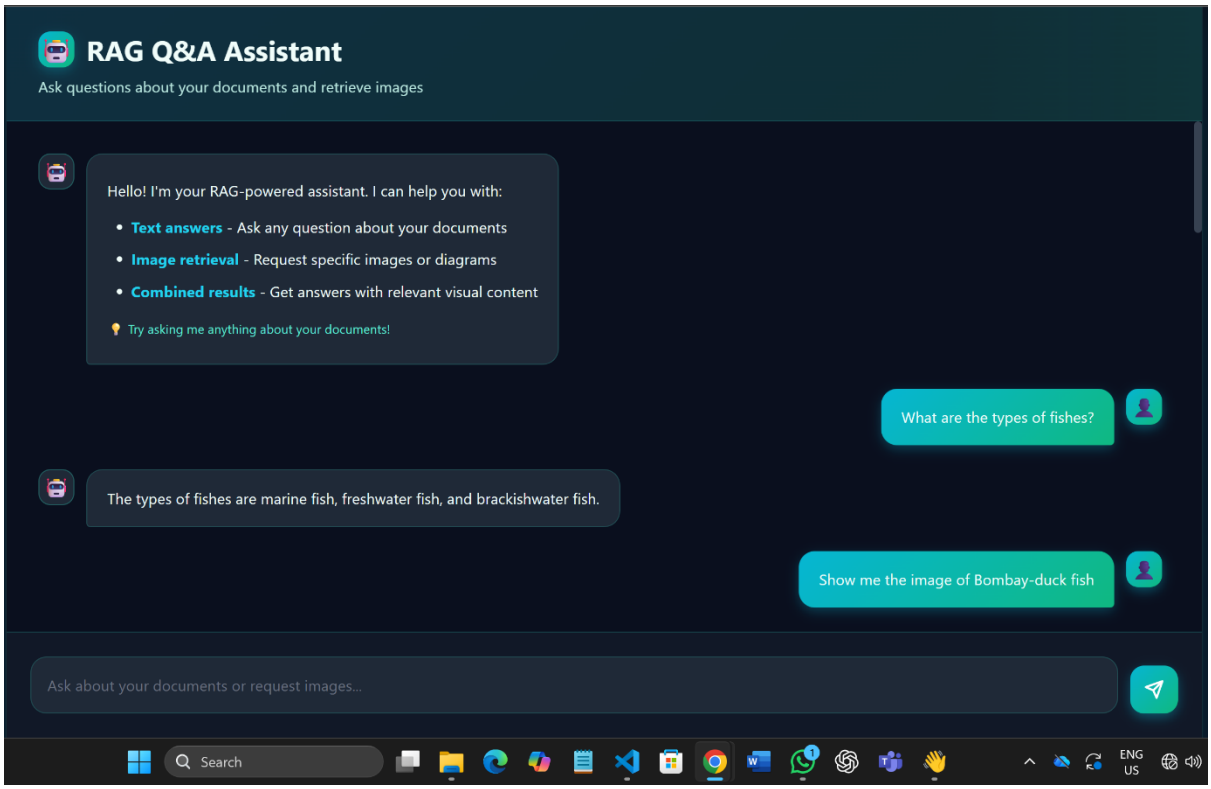

Figure 4: Implementation Screenshots

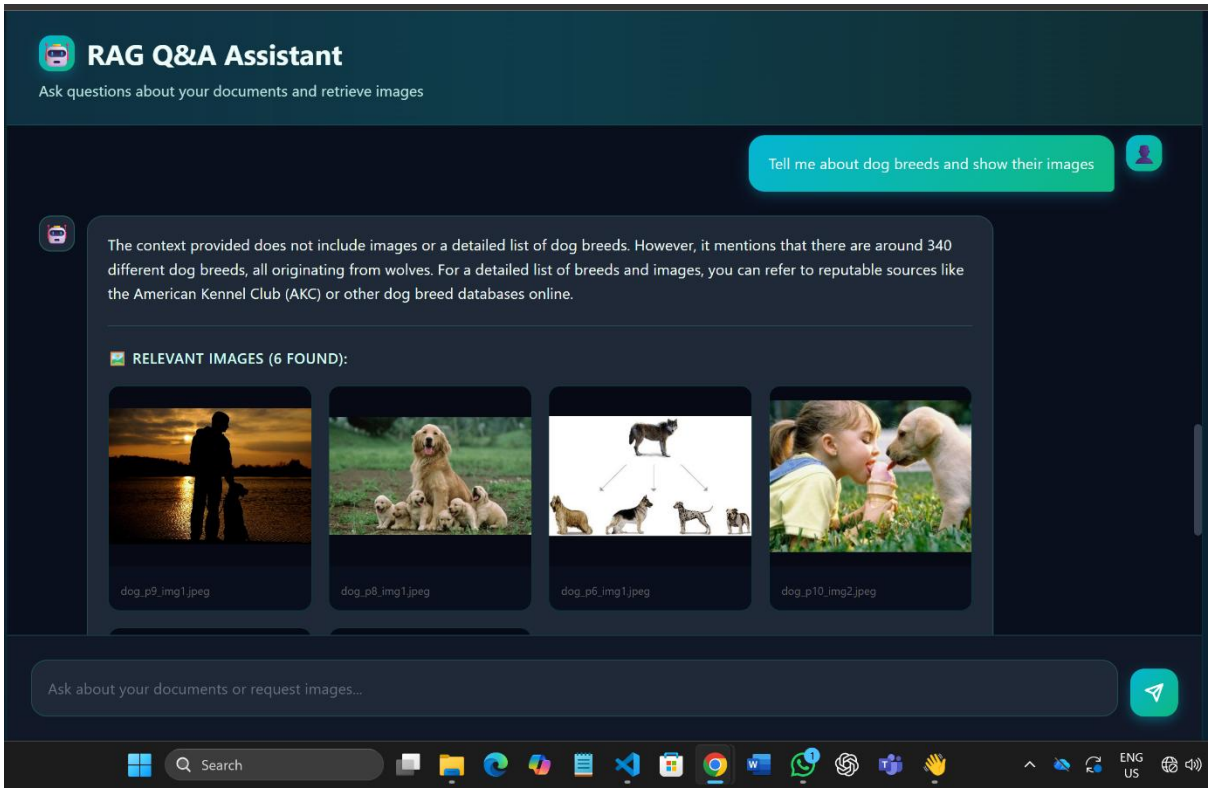

Figure 5: Implementation Screenshots

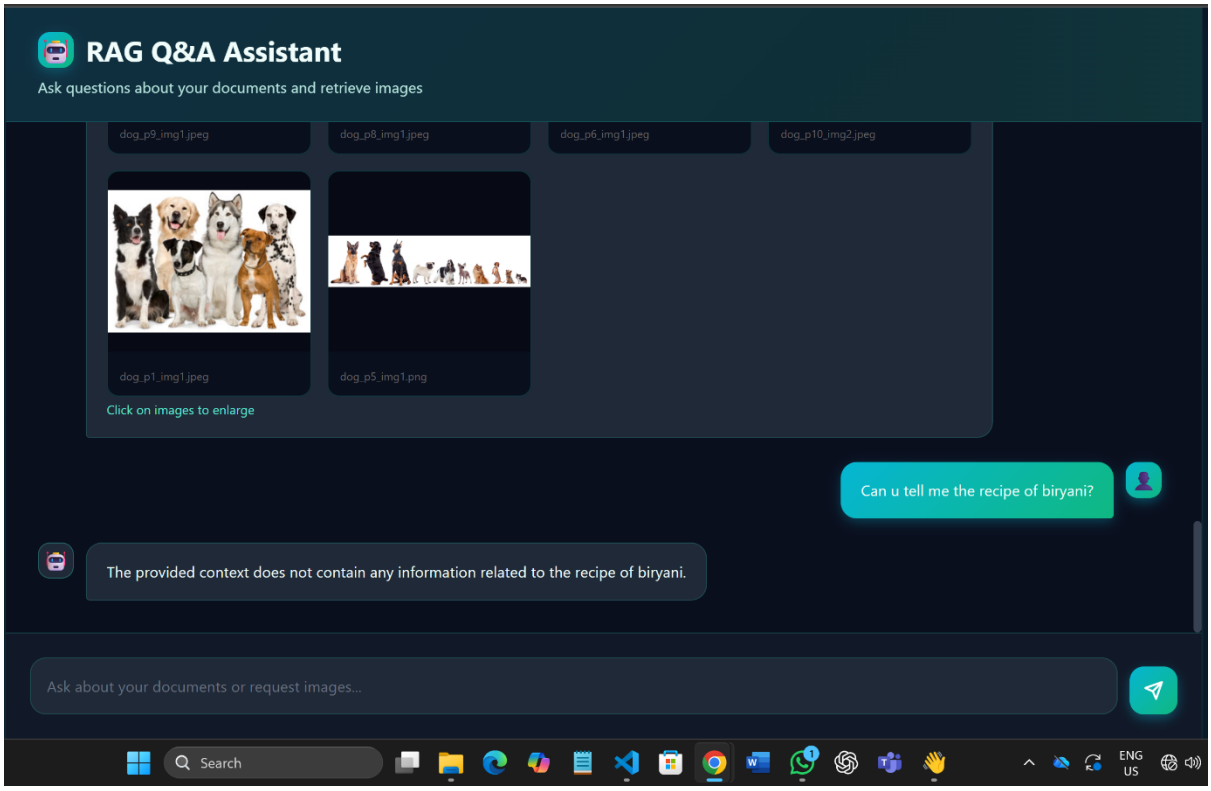

Figure 6: Implementation Screenshots

7. Results and Discussion

The implemented Offline RAG Q&A system successfully integrates multilingual text and image retrieval from a local PDF-based knowledge base, demonstrating privacy-preserving semantic search and multimodal response generation. The ingestion pipeline, powered by LangChain and FAISS with multilingual sentence transformer embeddings, effectively processes PDF content by extracting text chunks with page metadata and images with their surrounding contextual text — enabling queries in over 50 languages to retrieve semantically relevant results, including images associated with textual context.

Upon querying, the system dynamically analyses the intent of the input using multilingual keyword-based heuristics across 12 languages to determine whether the user requires text, images, or a combination of both. This multilingual intent detection is a significant advancement over the earlier single-language approach. The FAISS vector store efficiently returns the top-k relevant documents, and the local Jan LLM generates concise, context-aware answers from retrieved text. The frontend seamlessly integrates image results into the chat interface, with interactive features such as image enlargement and clickable citations, ensuring a user-friendly experience.

Testing demonstrates that the system correctly handles diverse query types. Text-only queries (e.g., 'What are the types of fishes?') receive concise textual answers grounded in the uploaded documents. Image-retrieval queries (e.g., 'Show me the image of Bombay-duck fish') return relevant image grids from PDFs. Combined queries (e.g., 'Tell me about dog breeds and show their images') receive both textual explanations and associated image results. Out-of-context queries (e.g., 'Can you tell me the recipe of biryani?') are handled gracefully with a response indicating the information is not available in the uploaded documents — demonstrating effective retrieval grounding and hallucination prevention.

The offline voice input feature, powered by faster-whisper, successfully transcribes queries locally after the one-time model download. The system correctly routes voice-transcribed queries through the same multilingual intent analysis and retrieval pipeline, confirming end-to-end voice query functionality without internet connectivity.

Multilingual query support was verified with queries in multiple languages. The paraphrase-multilingual-MiniLM-L12-v2 model correctly embedded multilingual queries into the same semantic space as the English-embedded document content, enabling cross-lingual retrieval with reasonable accuracy for semantically equivalent queries.

While the system performs well for small to medium-sized knowledge bases, some limitations were observed. Image extraction accuracy depends on PDF structure — images embedded as vector graphics or within complex page layouts may not be extracted accurately. The text-based contextual embedding for images (rather than pixel-level embedding) means that images without surrounding descriptive text may not be retrieved for relevant visual queries. Additionally, the current full-rebuild approach on every ingestion run means that adding a single new PDF to a large document collection requires reprocessing all documents. Future improvements could include incremental index updates, fine-tuning embeddings on domain-specific multilingual data, and integrating an OCR module for scanned PDFs.

Overall, the project demonstrates the feasibility of a fully offline, privacy-preserving multimodal RAG system combining multilingual semantic search, contextual image retrieval, offline voice input, and local LLM inference — providing a robust, interactive knowledge assistant that operates entirely on-device.

8. Conclusion

The developed Offline Multimodal RAG system successfully demonstrates an effective integration of multilingual text and image retrieval from a local PDF-based knowledge repository, operating entirely without internet connectivity during operation. By leveraging multilingual sentence transformer embeddings (paraphrase-multilingual-MiniLM-L12-v2), FAISS-based vector search, contextual image extraction via PyMuPDF, offline voice input via faster-whisper, and local LLM inference through Jan AI, the system is capable of providing precise, context-aware responses to user queries across 50+ languages — while ensuring that all data remains on the user's device.

The project highlights the potential of privacy-preserving multimodal retrieval systems, demonstrating that combining multilingual text embeddings with contextual image extraction can significantly improve the relevance and accessibility of responses across diverse language communities. The addition of offline voice input further lowers barriers for non-keyboard users and establishes a foundation for hands-free document querying. Source citations with clickable links to originating PDFs add transparency and trust to the system's responses.

Despite minor limitations in image caption extraction accuracy for complex PDF layouts and the full-rebuild requirement on every ingestion run, the system performs reliably for small to medium knowledge bases and gracefully handles out-of-context queries without hallucination.

In conclusion, this work validates the feasibility of creating an interactive, privacy-first knowledge assistant that efficiently bridges multilingual semantic text search with contextual image retrieval and offline voice input — providing a robust foundation for future improvements such as incremental index updates, OCR for scanned PDFs, domain-specific embedding fine-tuning, and scalability to larger multilingual document collections.

9. Future Scope

The RAG system developed in this project can be further enhanced in multiple directions:

1. Incremental Index Updates: The current implementation performs a full vector store rebuild on every ingestion run. Implementing incremental updates — adding only new or modified PDFs to the existing index without reprocessing unchanged documents — would significantly reduce ingestion time for large document collections.
2. OCR for Scanned PDFs: Integrating an OCR module (e.g., Tesseract or a neural OCR model) would enable the system to process scanned PDFs and image-only pages that currently produce no extractable text.
3. Domain-Specific Embedding Fine-Tuning: Embedding models and LLMs can be fine-tuned on domain-specific corpora (medicine, law, engineering) to improve retrieval accuracy and response quality for technical subject matter.
4. Support for Additional File Types: Expanding beyond PDFs to include Word documents, Excel sheets, PowerPoint presentations, and web content would make the system more versatile for diverse enterprise use cases.
5. Advanced Image Understanding with Visual Models: Integrating vision-language models (e.g., LLaVA, MiniGPT-4) for pixel-level image understanding would enable richer visual retrieval beyond text-based contextual embedding — particularly for diagrams, charts, and technical figures without adjacent text.
6. Conversational Context Retention: Implementing multi-turn conversation memory would allow the chatbot to reference previous messages within a session, enabling follow-up questions and iterative refinement of answers.
7. Scalability and Cloud-Optional Deployment: While designed for fully local use, adding an optional cloud deployment mode with distributed vector stores and GPU acceleration would support organisations with larger document collections and heavier concurrent usage.
8. Real-Time Index Updates: Implementing file system watchers to automatically detect new, modified, or deleted PDFs and trigger incremental ingestion would eliminate the need for manual re-ingestion triggers.
9. Expanded Multilingual Voice Support: Extending voice input to support language-specific Whisper models (medium, large) for improved transcription accuracy in low-resource languages beyond the base model's capabilities.

10. References

1. Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., ... & Amodei, D. (2020). Language Models are Few-Shot Learners. Advances in Neural Information Processing Systems, 33, 1877-1901.
2. OpenAI. (2023). OpenAI API Documentation. Retrieved from https://platform.openai.com/docs
3. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP.
4. Johnson, J., Douze, M., & Jegou, H. (2019). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data, 7(3), 535-547.
5. LangChain Community. (2024). LangChain Community Vectorstores Documentation. Retrieved from https://www.langchain.com
6. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... & Chintala, S. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. Advances in Neural Information Processing Systems, 32.
7. Hugging Face. (2024). Sentence Transformers Library. Retrieved from https://www.sbert.net
8. MuPDF. (2024). PyMuPDF Documentation. Retrieved from https://pymupdf.readthedocs.io
9. Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2022). Robust Speech Recognition via Large-Scale Weak Supervision (Whisper). OpenAI Technical Report.
10. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is All You Need. Advances in Neural Information Processing Systems, 30, 5998-6008.
11. Chen, D., Fisch, A., Weston, J., & Bordes, A. (2017). Reading Wikipedia to Answer Open-Domain Questions. ACL 2017.
12. Wang, S., Pan, Y., & Li, Z. (2022). Enhancing RAG Systems with Contextual Image Understanding. Journal of Artificial Intelligence Research, 74, 105-125.
13. Jan AI. (2024). Jan - Open-source local AI. Retrieved from https://jan.ai
14. faster-whisper. (2024). faster-whisper: Faster Whisper transcription with CTranslate2. Retrieved from https://github.com/SYSTRAN/faster-whisper